

ECE 220 Computer Systems & Programming

Lecture 6 — Introduction to C: Control Structures, Basic I/O

September 10, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

 **ILLINOIS**

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

The Structure Problem

**Every interesting program decides and repeats.
In LC-3 that meant branches and labels — C gives them
shape.**

```
LOOP  ADD R2, R2, #-1  
      BRz DONE  
      ADD R0, R0, R1  
      BR  LOOP  
DONE  ...
```

branches + labels (Lecture 1)



```
if (cond) {...} else {...}
```

```
while (cond) {...}
```

```
for (init; cond; update) {...}
```

structured control (today)

Today's Two Questions

**You write the structure —
the compiler writes the branches.**

1. How do C's decisions and loops map to the LC-3 branches you have been writing by hand?
2. Which construct fits which job — and what goes wrong (fall-through, infinite loops)?

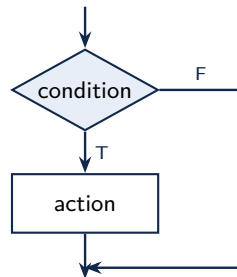
Plus: the rest of `printf/scanf` — reading two values, hex in and out, and rounding.

The if Statement (BR, Structured)

```
int x = 14, y = 0, month = 4;
/* --- if ----- */
if (x < 0)
    x = -x;           /* negate only if negative */
/*
if ((x > 10) && (x < 20)) {
    y = x % 4;        /* y = ? */
    printf("y = %d\n", y);
}
```

In-class: with $x = 14$, what does this print?

$y =$ _____

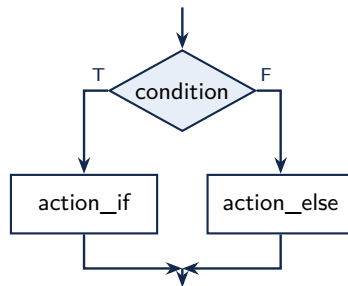


The compiler turns the condition into exactly the BRn/BRzp tests you wrote in LC-3.

The if-else Statement

```
/* --- if-else ----- */  
if (x % 2 == 0)  
    printf("%d is even\n", x);  
else  
    printf("%d is odd\n", x);
```

- Exactly one of the two branches runs
- `else` binds to the *nearest* unmatched `if` — braces prevent surprises



The switch Statement

```
/* --- switch ----- */
switch (month) {
    case 12: case 1: case 2:
        printf("winter\n");
        break;          /* without it: fall-
through! */
    case 3: case 4: case 5:
        printf("spring\n");
        break;
    default:
        printf("summer or fall\n");
}
```

- Cleaner than a chain of if ... else if ... else
 - Stacked labels (case 12: case 1: case 2:) act as or
 - Without break, execution
-

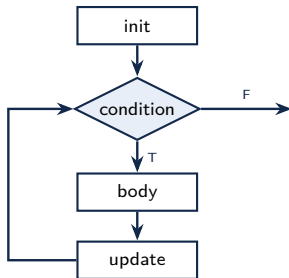
while and do-while

```
/* --- while: may run zero times ----- */
x = 0;
while (x < 10) {
    printf("x=%d\n", x);
    x++;
}
/* --- do-while: runs at least once ----- */
x = 0;
do {
    printf("x=%d\n", x);
    x++;
} while (x < 10);
```

- **while:** test first — body may run _____ times
- **do-while:** test last — body runs _____
- Use do-while when you must act *before* you can test (read input, then check it)

The for Statement

```
/* --- for: init; condition; update -----  
*/  
for (x = 0; x < 10; x++) {  
    if (x == 5)  
        break;           /* try: continue */  
    printf("x=%d\n", x);  
}
```



In-class:

- Output of the loop as written?

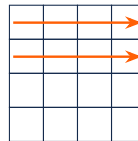
- Replace `break` with `continue`?

- What makes a loop **infinite**?

Nested Loops

Inner loop runs to completion once per outer iteration — the shape of every table, grid, and matrix:

```
for (row = 0; row < N; row++) {  
  for (col = 0; col < N; col++) {  
    /* visit (row, col) */  
  }  
  /* end of one row */  
}
```



You did this in LC-3: the print-hex program — inner loop shifts 4 bits to build one digit, outer loop prints the 4 digits.

printf and scanf, in Practice

```
/* Basic I/O practice */
#include <stdio.h>
int main() {
    int len, ht, num;
    printf("%+d\n", 43);           /* forced sign */
    printf("%x\n", 43 + 59);       /* hex output */
    printf("%.3f\n", 22.0 / 7);    /* 3 decimals */
    scanf("%d %d", &len, &ht);     /* two values */
    scanf("%x", &num);             /* hex input! */
    printf("area=%d num=%d\n", len * ht, num);
    return 0;
}
```

In-class — predict the output:

- `%+d` of 43: _____
- `%x` of 43 + 59: _____
- `%.3f` of 22.0/7: _____
- One `scanf` can read *several* values
- `%x` parses hex input: typing `ef` reads _____
- `man 3 printf` is the reference

That `&` in front of every `scanf` variable? Next week it stops being magic — pointers.

In-Class Problem: Identity Matrix

Print an $N \times N$ identity matrix using nested loops:

```
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

- When does a cell hold 1?

- Where does the newline go?

```
/* Print an N x N identity matrix with nested loops */
#include <stdio.h>
#define N 5

int main() {
    /* declare loop counters */

    /* outer loop: one pass per row */

        /* inner loop: one pass per column */

            /* print 1 on the diagonal, 0 elsewhere */
            */

            /* end the row with a newline */

    return 0;
}
```

Follow-Up Questions

1. Read the matrix size n from the user — which function, and which construct repeats the prompt until $0 < n < 10$? _____
2. Stop after printing the *third* 1 on the diagonal — why isn't one `break` enough?

Sketch your loop structure here — then we build `followups.c` together.

Summary & What's Next

Today

- **Decisions:** `if`, `if-else`, `switch` (mind the `break` — fall-through is the classic bug)
- **Loops:** `while` (may skip), `do-while` (runs once for sure), `for` (init; condition; update); `break` and `continue`
- **Nested loops** traverse grids — rows outside, columns inside
- **I/O:** format specifiers work on input too; `scanf` reads several values at once

Next lecture

- Functions in C — the return of the subroutine, with types

Code from today: `decisions.c`, `loops.c`, `io_demo.c`, `identity.c`, `followups.c`.